

MAQUETACIÓN: SITIOS WEB INCLUSIVOS

Preprocesadores CSS, responsividad, accesibilidad y frameworks

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Miguel Ángel Muñoz

Contenido

1. Preprocesadores CSS	3
1.1. Variables y nesting	3
1.2. Mixins, funciones y módulos	3
2. Responsividad	4
2.1. Breakpoints y media queries	4
2.2. Mobile first	4
3. Accesibilidad	5
4. Frameworks	5
5. Qué hemos aprendido	6

1. Preprocesadores CSS

Un **preprocesador CSS** permite escribir hojas de estilo con una sintaxis ampliada —con variables, anidación, funciones— que después se compila a CSS estándar, legible por cualquier navegador. Entre los más conocidos están **SASS**, **LESS**, **Stylus** y **PostCSS**; este documento se centra en SASS, aunque los conceptos son fácilmente trasladables a cualquiera de los otros.

1.1. Variables y nesting

A diferencia de las variables nativas de CSS (**--mi-variable**), que se resuelven en el navegador y dependen del elemento donde se declaren, las variables de SASS (**\$mi-variable**) se compilan antes de llegar al CSS final y pueden definirse de forma global, manteniendo el último valor que se les haya asignado.

```
$color-primario: #2331d4;

.boton {
  color: $color-primario;
}
```

El **nesting** (anidación) evita repetir una y otra vez el mismo selector padre cuando hay que dar estilos a varios elementos hijos, y usa el símbolo **&** para referenciar al elemento padre en combinaciones con pseudoclases:

```
#main-menu {
  color: black;
  h2 { font-size: 20px; }
  a {
    text-decoration: none;
    &:hover { text-decoration: underline; }
  }
}
```

1.2. Mixins, funciones y módulos

Los **mixins** (**@mixin** / **@include**) empaquetan bloques de estilo reutilizables, admitiendo parámetros obligatorios u opcionales; las **funciones** (**@function**) se les parecen, pero están pensadas para devolver un valor en lugar de aplicar estilos directamente.

```
@mixin flex-center {
  display: flex;
  justify-content: center;
  align-items: center;
}

.contenedor {
  @include flex-center;
}
```

Para organizar proyectos grandes, SASS se apoya en módulos: @use carga otro archivo SASS, @forward expone su contenido para que otros lo utilicen, y @import combina varios archivos en uno solo.

★ Lo más importante

- ▶ Las variables SASS se fijan en tiempo de compilación; las variables CSS nativas, en el navegador.
- ▶ El nesting con & reduce la repetición de selectores padre.
- ▶ Mixins y funciones son la base para no repetir código entre distintos estilos.

2. Responsividad

Diseñar de forma responsive exige perseguir dos objetivos a la vez: **flexibilidad** (huir de medidas fijas y dejar que los elementos se adapten al espacio disponible) y **adaptabilidad** (introducir cambios reales de estructura según el tamaño de pantalla, no solo estirar lo mismo).

2.1. Breakpoints y media queries

Para decidir en qué puntos cambia el diseño se definen unos breakpoints o puntos de corte, que suelen actuar como un contrato entre diseño y desarrollo:

sm — 576px (móviles) · **md** — 768px (tabletas) · **lg** — 993px (monitores pequeños) · **xl** — 1200px (monitores estándar) · **xxl** — 1400px (monitores grandes)

Esos puntos de corte se aplican mediante **media queries**, que permiten definir reglas de estilo según el ancho del visor (u otras características del medio, ya que un "medio" no tiene por qué ser una pantalla: también puede ser un documento impreso):

```
@media screen and (min-width: 576px) {
  .caja { flex-direction: row; }
}
```

2.2. Mobile first

La metodología **mobile first** propone invertir el orden habitual de trabajo: en lugar de diseñar primero para escritorio y luego adaptar hacia abajo, se diseñan y construyen primero todos los estilos pensando en la pantalla de un móvil, y solo después se añaden los breakpoints y media queries necesarios para pantallas mayores.

★ Lo más importante

- ▶ Flexibilidad (medidas adaptables) y adaptabilidad (cambios reales por tamaño) son objetivos distintos y complementarios.
- ▶ Los breakpoints típicos son 576, 768, 993, 1200 y 1400px.
- ▶ Mobile first: diseña primero para móvil y añade breakpoints hacia pantallas mayores.

3. Accesibilidad

La **accesibilidad** busca que cualquier persona, incluidas las que tienen algún tipo de discapacidad visual o de otro tipo, pueda usar un sitio web sin barreras. No es un añadido opcional: debería ser un requisito más de cualquier proyecto, especialmente en el sector público.

La **WCAG** define tres niveles de cumplimiento —A, AA y AAA—, siendo AA el mínimo que suelen exigir las administraciones públicas. Dos herramientas ayudan a comprobarlo en la práctica: **Lighthouse** (integrada en las DevTools, mide también SEO y rendimiento) y **Axe**, especializada en accesibilidad y con la ventaja de poder incluirse en tests automáticos.

Algunas buenas prácticas concretas marcan buena parte de la diferencia: cuidar el contraste de color, añadir siempre un alt descriptivo a las imágenes, incluir *captions* en el contenido audiovisual, mantener una estructura semántica correcta, declarar el atributo lang en la etiqueta <html> y asociar correctamente las *labels* de los formularios con sus campos.

Los enlaces (<a>) deben usarse únicamente como enlaces y diferenciarse claramente del resto del contenido: reutilizarlos como si fueran botones confunde tanto a las personas usuarias como a los lectores de pantalla.

★ Lo más importante

- ▶ La WCAG define tres niveles: A, AA (mínimo habitual) y AAA.
- ▶ Lighthouse y Axe son las herramientas de referencia para auditar accesibilidad.
- ▶ Contraste, alt, estructura semántica y formularios correctos cubren buena parte del trabajo.

4. Frameworks

Un **framework** es, según se explica habitualmente, como una caja de herramientas que ya trae funcionalidades resueltas: la diferencia con una simple librería es que obliga a seguir su estructura y sus convenciones.

Bootstrap es el más extendido y conocido: muy personalizable y con una enorme cantidad de componentes ya resueltos. **Tailwind**, más reciente, sigue un enfoque distinto: expone multitud de clases de utilidad que se aplican directamente en el HTML, dando más control granular a cambio de escribir más clases.

Para aprender de verdad a maquetar, conviene practicar primero sin ningún framework: entender por qué existen y qué resuelven exactamente ayuda a elegirlos con criterio más adelante, en lugar de depender de ellos desde el primer día.

5. Qué hemos aprendido

En este documento hemos visto cómo los preprocesadores como SASS añaden lógica —variables, nesting, mixins, funciones— a un lenguaje tan declarativo como CSS; cómo lograr que un sitio sea realmente responsive combinando flexibilidad, adaptabilidad y una estrategia mobile first; por qué la accesibilidad no es negociable y qué herramientas ayudan a verificarla; y qué aportan (y qué exigen a cambio) los frameworks CSS más utilizados.

La maquetación profesional no consiste en dominar un framework concreto, sino en entender los fundamentos que hay debajo: solo así es posible tomar buenas decisiones cuando el framework de turno no encaja con lo que el proyecto necesita.

¡Enhorabuena! Documento completado